Python Concurrency Models

threads, processes, asyncio and the GIL

Watch the 4-min video

In one line: match the tool to the work: asyncio for many I/O waits, threads for blocking I/O libraries, processes for CPU-bound Python (the GIL lets one thread run Python bytecode at a time per process), and task queues for heavy jobs; one blocking call freezes an event loop, so offload CPU work to a process pool, and run several worker processes per server.

1 One endpoint stalls them all

Your FastAPI service handles a thousand requests per second for database lookups. Then you add PDF parsing to one endpoint, and the whole service grinds to a halt, even the fast endpoints. Same code, same server. The problem is the wrong concurrency model.

2 First: what kind of work?

I/O-bound work spends most of its time **waiting**: for the network, the database, a disk. **CPU-bound** work spends its time **computing**: parsing, compressing, resizing images, running models. The right tool depends entirely on which one you have.

work	use	why
I/O-bound, many connections	asyncio	thousands of waits on one thread
I/O-bound, blocking library	threads (asyncio.to_thread)	GIL released while waiting
CPU-bound Python	processes (ProcessPoolExecutor)	one GIL per process → real parallelism
long / heavy jobs	task queue workers	off the request path entirely

Figure 1. Which concurrency tool fits which kind of work.

3 The GIL

The **global interpreter lock** (GIL) lets only one thread execute Python bytecode at a time in a process. Threads still help with I/O, because the GIL is **released while waiting** on the network or disk (and inside many C extensions like NumPy). But for CPU-bound Python code, threads don't run in parallel; they take turns. To use several cores for Python computation, you need **several processes**, each with its own interpreter and GIL.

4 asyncio, and the blocking trap

asyncio runs many tasks on one thread with an **event loop**. When a task `await`s I/O, the loop switches to another task. That's great for thousands of concurrent connections with very little memory. But one blocking call (a slow CPU loop, `time.sleep`, a synchronous library like `requests`) freezes **every** task on that loop. That's what happened with the PDF parsing.

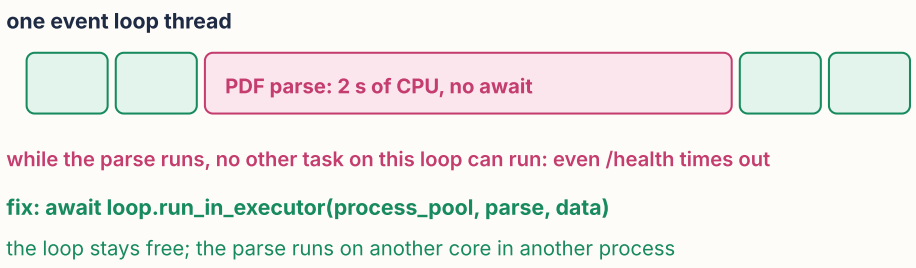


Figure 2. A CPU-heavy task without awaits blocks the event loop; every other request on that worker waits.

WORKED EXAMPLE: OFFLOADING PDF PARSING

The async API offloads CPU-heavy parsing to a process pool: `await loop.run_in_executor(pool, parse_pdf, data)` with a `ProcessPoolExecutor(max_workers=os.cpu_count())` created at startup. Each parse runs in its own process, on its own core, in parallel, while the event loop stays free to serve other requests. For blocking *I/O* libraries, `await asyncio.to_thread(func, …)` runs them in a thread pool instead.

5 In production: processes × event loops

Servers combine both. **Uvicorn** runs an asyncio event loop per worker; **Gunicorn** manages several worker processes, often with the Uvicorn worker class, typically one or two per CPU core. So processes give parallelism across cores, and asyncio gives high concurrency inside each process. For heavy jobs, use a task queue with separate workers (lesson 37).

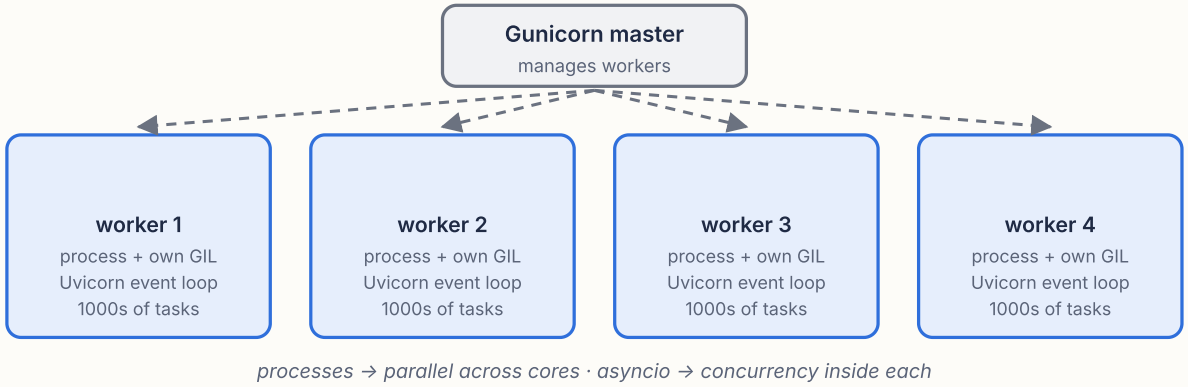


Figure 3. Gunicorn with Uvicorn workers: one process per core, each with its own event loop.

6 The future: free-threaded Python

Python 3.13 introduced an optional **free-threaded build** without the GIL (PEP 703), where threads can run Python code on several cores at once. It's experimental, and many libraries are still adapting. For now, design with the GIL in mind: processes for CPU, async or threads for I/O.

COMMON MISTAKES

- Calling `requests` or a sync DB driver inside `async def` routes.
- Using threads to speed up pure-Python number crunching.
- One Uvicorn worker on an 8-core machine.
- Creating a new process pool per request (huge startup cost).

7 Key terms

I/O-bound: time spent waiting on network or disk

CPU-bound: time spent computing

GIL: one thread runs Python bytecode at a time per process

Event loop: runs many tasks on one thread, switching at awaits

run_in_executor: run a function in a thread or process pool

asyncio.to_thread: run blocking code in a thread

Free-threaded build: Python without the GIL (3.13+, experimental)

8 Interview questions

Q1. When does asyncio help and when not? BASIC

It helps for I/O-bound work with many concurrent connections, like API calls and database queries; it doesn't help CPU-bound work, and any blocking call freezes the whole loop, so offload CPU work to a process pool.

Q2. How does the GIL affect scaling? BASIC

Only one thread runs Python bytecode at a time per process, so threads don't parallelise CPU-bound Python, and you scale across cores with multiple processes or workers.

Q3. A FastAPI endpoint resizes images and slows the whole service. Fix it. SCENARIO

Move resizing to a ProcessPoolExecutor via run_in_executor, or to a task queue with dedicated workers; keep the event loop for I/O only.

Q4. How many Gunicorn workers for a 4-core box running an async API? SCENARIO

Start with about one to two Uvicorn workers per core (4 to 8), then load test: more workers help only if the loop is busy; memory per worker is the other limit.

Q5. Do threads help with NumPy-heavy code? TRICKY

Often yes: NumPy releases the GIL inside its C loops, so threads can run those parts in parallel. Pure-Python loops don't benefit.

Q6. Why is a sync database driver in an async route so harmful? TRICKY

Each query blocks the event loop thread for its whole duration, so the worker serves one request at a time instead of thousands; use an async driver or to_thread.

Next: SD-J-006 · Batching backendroadmap.com